

Génération procédurale de terrain aléatoire : l'équilibre entre performance et cohérence

Des jeux comme Minecraft[1] ou No Man's Sky mettent en évidence la puissance de la génération procédurale, souvent qualifiée d'« aléatoire contrôlé ». Cette approche permet de varier chaque partie de jeu tout en conservant des environnements cohérents, ce qui m'a particulièrement intéressé par son équilibre entre hasard et maîtrise.

Certains algorithmes de génération procédurale partent d'un ou plusieurs points initiaux pour créer le reste d'un terrain. De plus, à partir d'un terrain existant, ils peuvent aussi générer des zones adjacentes à ce terrain, permettant une extension progressive et cyclique du terrain généré.

Le candidat atteste avoir travaillé en monôme.

Positionnement thématique (ÉTAPE 1) :

- INFORMATIQUE (*Informatique pratique*)
- INFORMATIQUE (*Informatique Théorique*)

Mots-clés (ÉTAPE 1) :

Mots-clés (en français)	Mots-clés (en anglais)
Génération procédurale	Procedural generation
Carte de hauteur	Heightmap
Bruit de Perlin	Perlin noise
Déplacement du point médian	Midpoint displacement
Algorithme Diamant Carré	Diamond-square algorithm

Bibliographie commentée

L'industrie du jeu vidéo utilise fréquemment le principe de **génération procédurale**, qui permet de créer un grand nombre de contenu à partir d'algorithmes, sans ajout manuel spécifique du développeur. Cette approche favorise une forte rejouabilité tout en limitant les coûts de conception. L'un des premiers jeux à exploiter pleinement ce principe est **Rogue** [2], sorti en 1980, qui génère de manière aléatoire mais cohérente des salles de donjons. De

nombreux jeux ont ensuite repris cette idée avec succès, notamment Minecraft [1] ou No Man's Sky, capables de générer jusqu'à 2 puissance 64 mondes différents, soit environ 18 milliards de milliards de mondes, illustrant l'efficacité et la robustesse de ces méthodes.

Parmi les algorithmes les plus utilisés dans l'industrie figure le **bruit de Perlin** [7], conçu par **Ken Perlin** et formalisé dans une publication de 1985. Cet algorithme vise à produire un bruit pseudo-aléatoire continu, permettant d'obtenir des textures et des reliefs visuellement naturels. Ken Perlin développa initialement cette méthode en 1982, motivé par sa frustration face au caractère trop artificiel des images générées par ordinateur à l'époque. Le bruit de Perlin permit notamment la création d'effets spéciaux crédibles, utilisés dans le film Tron [6] de Steven Lisberger, sorti en 1982.

Toutefois, le bruit de Perlin présente certaines limites en termes de performances, en particulier lorsque la *dimension du bruit* augmente. Afin d'y remédier, Ken Perlin proposa une version optimisée : le **bruit Simplex** [9]. Cet algorithme permet de générer un bruit de qualité comparable, tout en réduisant la **complexité de calcul**, ce qui le rend plus adapté à la génération de terrains de grande taille ou de dimensions élevées.

Parallèlement à ces méthodes basées sur le bruit, une autre approche de génération procédurale existe : l'**algorithme de déplacement du point médian**. Cette méthode, initialement théorisée par trois auteurs [4], permet de générer de manière **itérative** un terrain en une dimension. Le principe peut être généralisé à deux ou trois dimensions afin de créer des **cartes de hauteurs**, qui représentent un terrain à partir de son altitude. Cette représentation discrète est particulièrement adaptée à la modélisation numérique des reliefs.

Ainsi, ces deux approches — génération par **bruit pseudo-aléatoire** et génération par **déplacement du point médian** — constituent des méthodes concurrentes pour produire des terrains aléatoires mais cohérents. Les exemples issus de l'industrie du jeu vidéo montrent qu'il est possible d'atteindre une génération efficace à grande échelle, ce qui conduit à s'interroger sur les contraintes nécessaires pour obtenir à la fois **cohérence**, **réalisme** et **rapidité de calcul**.

Concernant l'implémentation du bruit de Perlin, un guide en ligne rédigé par deux auteurs [3] aborde cette méthode sous l'angle des "*fragment shaders*", offrant une approche pratique et orientée performance. Cette vision est complétée par les travaux de **Paul Bourke** [8], qui recense différentes techniques de génération de bruit et en analyse les spécificités, notamment celles du bruit de Perlin.

Dans le cadre de l'autre approche, **Paul Martz** [5] décrit l'utilisation du déplacement du point médian pour implémenter l'**algorithme Diamant-Carré**, qui se distingue par son fonctionnement à la fois **itératif et récursif**, et qui est largement utilisé pour la génération de cartes de terrains en 2 dimensions.

Problématique retenue

Comment générer, à l'aide d'algorithmes pseudo-aléatoires, un terrain **cohérent et réaliste** tout en garantissant une **efficacité de calcul** suffisante pour une génération **rapide** et **paramétrable** ?

Objectifs du TIPE du candidat

- D'abord, choisir l'algorithme de génération le plus adaptée aux différentes contraintes
- > Notamment, entre le **Bruit de Perlin** et l'**Algorithme Diamant-Carré**
- Ensuite, implémenter une version de cet algorithme qui génère du terrain
- Puis, optimiser les paramètres de cet algorithme pour obtenir la meilleure génération
- Enfin, éventuellement implémenter un autre algorithme pour pouvoir comparer les différents résultats selon les paramètres

Références bibliographiques (ÉTAPE 1)

- [1] ALAN ZUCCONI : Explication du fonctionnement du jeu Minecraft : www.alanzucconi.com/2022/06/05/minecraft-world-generation
- [2] WIKIPÉDIA : Page du jeu "Rogue" créé en 1980 : [en.wikipedia.org/wiki/Rogue_\(video_game\)](https://en.wikipedia.org/wiki/Rogue_(video_game))
- [3] PATRICIO GONZALEZ VIVO, JEN LOWE : The book of shaders - Guide à propos des "Fragment Shaders" : thebookofshaders.com
- [4] ALAIN FOURNIER, DON FUSSELL, LOREN CARPENTER : Computer Rendering of Stochastic Models : *Communications of the ACM Volume 25, (1982), 371 - 384*
- [5] PAUL MARTZ : Explication de l'algorithme Diamant-Carré : web.archive.org/web/20060420054134/http://www.gameprogrammer.com/fractal.html#diamond
- [6] IMDB : Cinématographie du film Tron : www.imdb.com/fr/title/tt0084827/
- [7] KEN PERLIN : An image synthesizer : *ACM SIGGRAPH Computer Graphics Volume 19, (1985), 287 - 296*
- [8] PAUL BOURKE : Bruit de Perlin et turbulences : paulbourke.net/fractals/noise/
- [9] KEN PERLIN : Noise Hardware : *In Real-Time Shading SIGGRAPH Course Notes, (2001)*